```
%cd /content
!git clone https://github.com/marcgpastor/03_MIAR_Proyecto.git repo_tmp
!mv repo_tmp/data /content/data
!rm -rf repo_tmp
```

```
/content
Cloning into 'repo_tmp'...
remote: Enumerating objects: 31, done.
remote: Counting objects: 100% (31/31), done.
remote: Compressing objects: 100% (27/27), done.
remote: Total 31 (delta 13), reused 13 (delta 3), pack-reused 0 (from 0)
Receiving objects: 100% (31/31), 31.89 KiB | 6.38 MiB/s, done.
Resolving deltas: 100% (13/13), done.
mv: cannot move 'repo_tmp/data' to '/content/data/data': Directory not empty
```

Algoritmos de optimización - Seminario

Nombre y apellidos: Marc González Pastor y Miguel Arribas Ramos
URL del repositorio: https://github.com/marcgpastor/03_MIAR_Proyecto
URL del notebook: https://colab.research.google.com/github/marcgpastor/03_MIAR_Proyecto/blob/master/proyecto.ipynb
Problema: Organizar los horarios de partidos de La Liga

0. Implementación

En este bloque se incluyen las funciones necesarias para cargar datos, calcular la audiencia y ejecutar los algoritmos.

```
from pathlib import Path
from collections import Counter
import numpy as np
import pandas as pd
```

```
REQ = [
    "equipos_categoria.csv",
    "audiencia_categoria.csv",
    "coef_horario.csv",
    "coef_coincidencia.csv",
]

DATA_DIR = Path("data")

def _load_equipos_categoria(path: Path) -> dict:
    """Devuelve un dict {equipo: categoria}."""
    df = pd.read_csv(path)
    df["equipo"] = df["equipo"].astype(str).str.strip()
    df["categoria"] = df["categoria"].astype(str).str.strip().str.upper()
    return dict(zip(df["equipo"], df["categoria"]))

def _load_audiencia_base(path: Path) -> dict:
    """Devuelve un dict {(cat1, cat2): audiencia_base} (en millones).
    Guardamos también la entrada simétrica (cat2, cat1)."""
    df = pd.read_csv(path)
    df["cat1"] = df["cat1"].astype(str).str.strip().str.upper()
    df["cat2"] = df["cat2"].astype(str).str.strip().str.upper()
    out = {}
    for r in df.itertuples(index=False):
        out[(r.cat1, r.cat2)] = float(r.audiencia_base)
        out[(r.cat2, r.cat1)] = float(r.audiencia_base)
    return out

def _load_coef_horario(path: Path) -> dict:
    """Devuelve un dict {slot: coef_horario}."""
    df = pd.read_csv(path)
    df["slot"] = df["slot"].astype(str).str.strip()
    df["coef_horario"] = df["coef_horario"].astype(float)
    return dict(zip(df["slot"], df["coef_horario"]))

def _load_coef_coincidencia(path: Path) -> dict:
    """Devuelve un dict {num_partidos_simultaneos: coef_multiplicativo}.
    Ejemplo: si num_partidos=2 -> coef=0.75 (cada partido se multiplica por 0.75)."""
    df = pd.read_csv(path)
    df["num_partidos"] = df["num_partidos"].astype(int)
    df["coef_coincidencia"] = df["coef_coincidencia"].astype(float)
    return dict(zip(df["num_partidos"], df["coef_coincidencia"]))

dict_equipos = _load_equipos_categoria(DATA_DIR / "equipos_categoria.csv")
dict_audiencia_base = _load_audiencia_base(DATA_DIR / "audiencia_categoria.csv")
dict_coef_horario = _load_coef_horario(DATA_DIR / "coef_horario.csv")
dict_coef_coincidencia = _load_coef_coincidencia(DATA_DIR / "coef_coincidencia.csv")
HORARIOS = list(dict_coef_horario.keys())

assert "V20" in dict_coef_horario and "L20" in dict_coef_horario
```

```
assert abs(dict_coef_coincidencia.get(1, 0.0) - 1.0) < 1e-12
```

```
def obtener_audiencia_base(partido) -> float:
    """Audiencia base (en millones) según categorías de los equipos."""
    equipo1, equipo2 = partido
    cat1, cat2 = dict_equipos[equipo1], dict_equipos[equipo2]
    return dict_audiencia_base[(cat1, cat2)]

def contar_partidos_por_horario(solucion) -> Counter:
    """solucion: dict {partido: slot} -> Counter {slot: k}"""
    return Counter(solucion.values())

def calcular_audiencia_partido(partido, slot, conteo_slots) -> float:
    """Aud(partido) = base * coef_horario(slot) * coef_coincidencia(k),
    donde k es el número de partidos simultáneos en esa franja."""
    base = obtener_audiencia_base(partido)
    coef_h = dict_coef_horario[slot]
    k = conteo_slots[slot]
    coef_c = dict_coef_coincidencia[k]
    return base * coef_h * coef_c

def calcular_audiencia_jornada(solucion, redondear=True) -> float:
    """Calcula la audiencia total de la jornada dada una asignación partido->slot."""
    conteo = contar_partidos_por_horario(solucion)
    total = sum(calcular_audiencia_partido(p, s, conteo) for p, s in solucion.items())
    return round(total, 2) if redondear else total
```

```
def asignar_partidos_jornada_fuerza_bruta(partidos) -> tuple:
    """Fuerza bruta (exacta). SOLO para casos pequeños (p. ej., 4 partidos).

    Recorre todas las asignaciones posibles y exige al menos un partido en
    V20 y al menos un partido en L20.
    """
    partidos = list(partidos)

    def backtrack(i, sol, conteo):
        if i == len(partidos):
            if conteo.get("V20", 0) == 0 or conteo.get("L20", 0) == 0:
                return None, float("-inf")
            return dict(sol), calcular_audiencia_jornada(sol)

        mejor_sol, mejor_val = None, float("-inf")
        partido = partidos[i]

        for slot in HORARIOS:
            sol[partido] = slot
            conteo[slot] = conteo.get(slot, 0) + 1

            s, v = backtrack(i + 1, sol, conteo)
            if v > mejor_val:
                mejor_sol, mejor_val = s, v

            conteo[slot] -= 1
            if conteo[slot] == 0:
                del conteo[slot]
            del sol[partido]

        return mejor_sol, mejor_val

    return backtrack(0, {}, {})
```

```
# Índices de las franjas obligatorias
IDX_V20 = HORARIOS.index("V20")
IDX_L20 = HORARIOS.index("L20")

# Máximo número de partidos simultáneos permitido por la tabla
max_partidos_por_horario = max(dict_coef_coincidencia.keys())


def evaluar_distribucion(distribucion_num_partidos_en_horarios, partidos_ordenados, audiencias_base_ordenadas) -> tuple:
    """Dada una distribución (cuántos partidos caen en cada slot), calcula:
      - la mejor audiencia total posible
      - una asignación concreta partido -> slot

    Para una distribución fija, el óptimo se obtiene emparejando
    audiencias base descendentes con multiplicadores descendentes.
    """
    multiplicadores = []
    for idx_slot, slot in enumerate(HORARIOS):
        k = distribucion_num_partidos_en_horarios[idx_slot]
        if k == 0:
            continue

        mult = dict_coef_horario[slot] * dict_coef_coincidencia[k]
        for _ in range(k):
            multiplicadores.append((mult, slot))
```

```python
    if len(multiplicadores) != len(partidos_ordenados):
        return float("-inf"), None

    multiplicadores.sort(key=lambda x: x[0], reverse=True)

    audiencia_total = 0.0
    asignacion = {}

    for partido, base, (mult, slot) in zip(partidos_ordenados, audiencias_base_ordenadas, multiplicadores):
        audiencia_total += base * mult
        asignacion[partido] = slot

    return audiencia_total, asignacion


def asignar_con_backtracking(indice_slot, restantes, distribucion, partidos_ordenados, aud_bases_ordenadas) -> tuple:
    """Reparte los partidos restantes entre todas las franjas."""
    if indice_slot == len(HORARIOS):
        if restantes != 0:
            return float("-inf"), None
        return evaluar_distribucion(distribucion, partidos_ordenados, aud_bases_ordenadas)

    mejor_aud = float("-inf")
    mejor_asig = None

    # Cuántos partidos extra podemos añadir en esta franja
    max_extra = min(restantes, max_partidos_por_horario - distribucion[indice_slot])

    for extra in range(max_extra + 1):
        distribucion[indice_slot] += extra

        aud, asig = asignar_con_backtracking(
            indice_slot + 1,
            restantes - extra,
            distribucion,
            partidos_ordenados,
            aud_bases_ordenadas
        )

        distribucion[indice_slot] -= extra

        if aud > mejor_aud:
            mejor_aud = aud
            mejor_asig = asig

    return mejor_aud, mejor_asig


def asignar_partidos_jornada(partidos_jornada) -> tuple:
    """Algoritmo exacto para una jornada, con al menos un partido en V20 y en L20."""
    aud_bases = [obtener_audiencia_base(p) for p in partidos_jornada]

    idx_ord = sorted(range(len(partidos_jornada)), key=lambda i: aud_bases[i], reverse=True)
    partidos_ord = [partidos_jornada[i] for i in idx_ord]
    aud_bases_ord = [aud_bases[i] for i in idx_ord]

    # Garantizamos al menos uno en V20 y al menos uno en L20
    distrib_inicial = [0] * len(HORARIOS)
    distrib_inicial[IDX_V20] = 1
    distrib_inicial[IDX_L20] = 1

    mejor_aud, mejor_asig = asignar_con_backtracking(
        0,
        len(partidos_jornada) - 2,
        distrib_inicial,
        partidos_ord,
        aud_bases_ord
    )

    return mejor_asig, round(mejor_aud, 2)
```

```python
def solucion_a_dataframe(solucion) -> tuple:
    """Devuelve (df, total) a partir del diccionario { (equipoA, equipoB): slot }."""
    conteo = Counter(solucion.values())
    rows = []

    for (eq1, eq2), slot in solucion.items():
        cat1, cat2 = dict_equipos[eq1], dict_equipos[eq2]
        base = dict_audiencia_base[(cat1, cat2)]
        coef_h = dict_coef_horario[slot]
        k = conteo[slot]
        coef_c = dict_coef_coincidencia[k]
        aud = base * coef_h * coef_c

        rows.append({
            "Partido": f"{eq1} - {eq2}",
```

```
            "Categorías": f"{cat1}-{cat2}",
            "Horario": slot,
            "Base": base,
            "CoefHorario": coef_h,
            "k_simult": k,
            "CoefCoinc": coef_c,
            "Audiencia": aud
        })

    df = pd.DataFrame(rows).sort_values(["Horario", "Partido"])
    df["Audiencia"] = df["Audiencia"].round(2)
    total = df["Audiencia"].sum().round(2)
    return df, total

def mostrar_resultado_jornada(solucion, algoritmo: str, nota: str | None = None) -> None:
    """
    Muestra de forma homogénea el resultado de una asignación partido -> franja.
    """
    df, total = solucion_a_dataframe(solucion)
    display(df)
    print(f"Algoritmo usado: {algoritmo}")
    if nota:
        print(nota)
    print(f"Total: {total} (millones)")
    print("Conteo por franja:", Counter(solucion.values()))
```

1. Descripción del problema

Desde la Liga de fútbol profesional se pretende organizar los horarios de los partidos de liga de cada jornada. Se conocen algunos datos que nos deben llevar a diseñar un algoritmo que realice la asignación de los partidos a los horarios de forma que maximice la audiencia.

- Los horarios disponibles se conocen a priori y son los siguientes:

Día	Horario
Viernes	20
Sábado	12, 16, 18, 20
Domingo	12, 16, 18, 20
Lunes	20

- En primer lugar, se clasifican los equipos en tres categorías según el numero de seguidores (que tiene relación directa con la audiencia). Hay 3 equipos en la categoría A, 11 equipos de categoría B y 6 equipos de categoría C.
- Se conoce estadísticamente la audiencia que genera cada partido según los equipos que se enfrentan (en horarios de máxima audiencia):

	Categoría A	Categoría B	Categoría C
Categoría A	2 M€	1.3 M€	1 M€
Categoría B		0.9 M€	0.75 M€
Categoría C			0.47 M€

- Si el partido no se disputa a las 20 horas del sábado o a la misma hora del domingo, la audiencia se reduce según los coeficientes de la siguiente tabla:

	Viernes	Sábado	Domingo	Lunes
12h	-	0.55	0.45	-
16h	-	0.7	0.75	-
18h	-	0.8	0.85	-
20h	0.4	1	1	0.4

- **Restricción**: se debe asignar **obligatoriamente** siempre un partido el **viernes** y un partido el **lunes**.
- La coincidencia de partidos en una misma franja horaria y en un mismo día es posible. Sin embargo, la audiencia de cada partido se verá afectada y se estima que se reduce en porcentaje según la siguiente tabla (dependiendo del número de coincidencias):

Coincidencias	-%
0	0%
1	25%
2	45%
3	60%
4	70%
5	75%
6	78%
7	80%
8	80%

> Nota de implementación: el fichero `coef_coincidencia.csv` proporciona el coeficiente en función de k (número de partidos simultáneos en la franja). Si se usa la tabla de "coincidencias" del enunciado (partidos adicionales), entonces

$k = \text{coincidencias} + 1.$

- Los cálculos asociados a una jornada de ejemplo se realizan según se muestra en la siguiente tabla:

Partido	Categorías	Horario	Base (M€)	Ponderación	Base*Ponderación	Corrección Coincidencia
Celta - Real Madrid	B-A	V20	1.3	0.4	0.52	0.52
Valencia - R.Sociedad	B-A	S12	1.3	0.55	0.72	0.72
Mallorca - Eibar	C-C	S16	0.47	0.7	0.33	0.33
Athletic - Barcelona	B-A	S18	1.3	0.8	1.04	1.04
Leganés - Osasuna	C-C	S20	0.47	1	0.47	0.47
Villarreal - Granada	B-C	D16	0.75	0.75	0.56	0.42
Alavés - Levante	B-B	D16	0.9	0.75	0.68	0.51
Espanyol - Sevilla	B-B	D18	0.9	0.85	0.77	0.77
Betis - Valladolid	B-C	D20	0.75	1	0.75	0.75
Atlético - Getafe	B-B	L20	0.9	0.4	0.36	0.36

2. Cuestiones a resolver

2.1. ¿Cuántas posibilidades hay sin tener en cuenta las restricciones?

Sin tener en cuenta restricciones, cada uno de los 10 partidos puede asignarse a cualquiera de las 10 franjas horarias. Por tanto, el número de asignaciones posibles es:

$$10^{10} = 10{,}000{,}000{,}000$$

(Diez mil millones de combinaciones).

2.2. ¿Cuántas posibilidades hay teniendo en cuenta todas las restricciones?

El enunciado indica: “Debemos asignar obligatoriamente siempre un partido el viernes y un partido el lunes”. Esta frase admite dos interpretaciones razonables, poniendo en evidencia las limitaciones del lenguaje natural:

Caso A) “Al menos uno” el viernes y “al menos uno” el lunes

Contamos todas las asignaciones posibles y restamos las que incumplen alguna de las dos condiciones (inclusión–exclusión):

- Total sin restricciones: 10^{10}
- Sin partido el viernes: 9^{10}
- Sin partido el lunes: 9^{10}
- Sin partido viernes **ni** lunes: 8^{10}

Por tanto:

$$N_A = 10^{10} - 2 \cdot 9^{10} + 8^{10} = 4{,}100{,}173{,}022$$

Caso B) “Exactamente uno” el viernes y “exactamente uno” el lunes

- Elegimos qué partido va en V20: 10 opciones.
- Elegimos qué partido va en L20: 9 opciones.
- Los 8 partidos restantes se asignan a las 8 franjas restantes (se permiten coincidencias): 8^8.

Por tanto:

$$N_B = 10 \cdot 9 \cdot 8^8 = 1{,}509{,}949{,}440$$

En este projecto, trabajaremos con el **Caso A (al menos uno)**.

2.3. ¿Cuál es la estructura de datos que mejor se adapta al problema?

Separaremos claramente:

- **Datos del problema (fijos)**: categorías de equipos, tabla de audiencia base, coeficientes por horario y por coincidencia.
- **Decisión (variable)**: a qué franja horaria asignamos cada partido.

Aunque puede parecer lógico usar un `DataFrame` (porque el enunciado está en forma de tabla), un `DataFrame` mezcla fácilmente datos fijos con decisiones y además no es la estructura más cómoda para generar o recorrer soluciones.

Por ello, representaremos una solución como un diccionario que asigna a cada partido una franja:

```
solucion = {
    ("Villarreal CF", "Deportivo Alavés"): "V20",
    ("Valencia CF", "Atlético de Madrid"): "S18",
    # ...
}
```

El `DataFrame` se reservará para mostrar los resultados de forma legible al final.

2.4. Según el modelo para el espacio de soluciones, ¿cuál es la función objetivo?

La función objetivo es **maximizar la audiencia total de la jornada**.

Para cada partido i:

$$\mathrm{Aud}(i) = \mathrm{AudBase}(\mathrm{cat}_i) \cdot \mathrm{CoefHorario}(\mathrm{slot}_i) \cdot \mathrm{CoefCoincidencia}(k_i)$$

donde k_i es el **número de partidos simultáneos** en la franja asignada al partido i (mismo día y misma hora).

La audiencia total a maximizar es:

$$\mathrm{Aud}_{\mathrm{total}} = \sum_i \mathrm{Aud}(i)$$

2.5. ¿Es un problema de maximización o minimización?

Es un problema de **maximización**, ya que buscamos **maximizar la audiencia total estimada** (en millones) de la jornada.

La dificultad principal es que la corrección por coincidencia introduce **dependencias entre decisiones**. La contribución de un partido no depende solo de su franja, sino también del **número de partidos asignados a esa misma franja**. Por tanto, no se puede optimizar cada partido de forma independiente.

2.6. Diseña un algoritmo para resolver el problema por fuerza bruta.

La fuerza bruta consiste en **probar todas las asignaciones posibles** de los P partidos a las H franjas horarias.

Esquema:

1. Generar recursivamente una asignación $slot(i) \in \{1, \ldots, H\}$ para cada partido i.
2. Comprobar la restricción de viernes y lunes.
3. Calcular la audiencia total de la asignación.
4. Quedarse con la mejor.

Pseudocódigo:

```
mejor_valor = -inf
mejor_sol = None

backtrack(i, asignacion):
  si i == P:
    si cumple_restriccion(asignacion):
      valor = audiencia_total(asignacion)
      actualizar mejor_valor, mejor_sol
    return

  para slot en 1..H:
    asignacion[i] = slot
    backtrack(i+1, asignacion)
```

(En el notebook se incluye una implementación en Python, pero solo es ejecutable en casos pequeños).

2.7. Calcula la complejidad del algoritmo por fuerza bruta.

En fuerza bruta, cada uno de los P partidos puede asignarse a cualquiera de las H franjas, por lo que el número de asignaciones es:

$$H^P$$

Para cada asignación, calcular la audiencia requiere:

- contar cuántos partidos caen en cada franja (para coincidencias): $O(P)$,
- sumar la contribución de los P partidos: $O(P)$.

Por tanto, evaluar una solución es $O(P)$ y la complejidad total es:

$$O\big(P \cdot H^P\big)$$

En nuestro caso $H = 10$ y $P = 10$, luego el orden es:

$$O(10 \cdot 10^{10}) \approx O(10^{11})$$

que es inasumible en tiempo.

Si además imponemos “exactamente 1 partido en V20 y 1 en L20”, el número de asignaciones se reduce a:

$$10 \cdot 9 \cdot 8^{P-2}$$

pero sigue siendo exponencial.

Complejidad espacial: $O(P)$ (profundidad de la recursión y estructura de la asignación).

2.8. Diseña un algoritmo que mejore la complejidad del algoritmo por fuerza bruta. Argumenta por qué crees que mejora el algoritmo por fuerza bruta.

La fuerza bruta asigna a cada partido una de las 10 franjas, lo que genera 10^{10} combinaciones (inviable).

La mejora se basa en separar el problema en dos niveles:

1) Distribución

Decidir cuántos partidos van en cada franja (solo cantidades), imponiendo:

- $V20 \geq 1$ y $L20 \geq 1$ (al menos un partido en viernes y lunes)
- la suma total de partidos en todas las franjas es $P = 10$
- se permiten coincidencias (una franja puede tener $k = 0, 1, 2, \ldots$ partidos), en la práctica hasta $k_{\max}$ según `coef_coincidencia.csv` (aquí $k_{\max} = 9$)

2) Asignación concreta (dada una distribución)

Una vez fijado cuántos partidos caen en cada franja, todos los partidos que comparten una franja con k partidos simultáneos comparten el mismo multiplicador:

$$m(\text{slot}, k) = \text{CoefHorario}(\text{slot}) \cdot \text{CoefCoincidencia}(k).$$

Como todos los términos son no negativos, para maximizar la suma $\sum_i \text{Base}_i \cdot m_i$ la asignación óptima se obtiene emparejando:

- partidos ordenados por audiencia base (descendente),
- con multiplicadores ordenados (descendente).

Esto es exacto (no heurístico). Para cada distribución calculamos su óptimo, y recorriendo todas las distribuciones válidas obtenemos el óptimo global para la jornada.

2.9. Calcula la complejidad del algoritmo.

Sea P el número de partidos (en este caso, $P = 10$) y sea H el número total de franjas (en este caso, $H = 10$).

Número de distribuciones

El backtracking recorre todas las formas de decidir cuántos partidos caen en cada franja, imponiendo:

- $V20 \geq 1$ y $L20 \geq 1$,
- y $\sum_{h=1}^{H} x_h = P$.

Equivalentemente, fijamos 1 partido en V20 y 1 en L20, y repartimos los $P - 2$ partidos restantes entre las $H = 10$ franjas (permitiendo que V20 y L20 también reciban más partidos). Sin considerar el límite $k_{\max}$, el número de distribuciones es el de las composiciones débiles:

$$N_{\text{dist}} = \binom{(P-2) + H - 1}{H - 1} = \binom{P+7}{9}.$$

Para $P = 10$ y $H = 10$:

$$N_{\text{dist}} = \binom{17}{9} = 24310.$$

(El límite $k_{\max}$ de `coef_coincidencia.csv` solo reduce este número.)

Coste de evaluar una distribución

Para cada distribución:

- construir la lista de multiplicadores (tamaño P): $O(P)$
- ordenar multiplicadores: $O(P \log P)$
- emparejar con las bases (ya ordenadas) y sumar: $O(P)$

Por tanto, evaluar una distribución cuesta $O(P \log P)$.

Complejidad total

Ordenar los partidos por base se hace una vez: $O(P \log P)$.

Luego:

$$T(P) = O\big(N_{\text{dist}} \cdot P \log P\big).$$

Con $H = 10$ fijo (caso del enunciado), esto es muy inferior a 10^{10} y para $P = 10$ es perfectamente asumible.

Complejidad espacial: $O(P)$ (listas de tamaño P y estructuras de asignación), más $O(H)$ por la recursión del backtracking.

2.10. Según el problema (y tenga sentido), diseña un juego de datos de entrada aleatorio.

Para validar el algoritmo sin depender de la jornada del enunciado, generamos una **jornada aleatoria** emparejando los 20 equipos (3 de categoría A, 11 de B y 6 de C) en 10 partidos sin repetición de equipos.

La generación se hace con una **semilla** para que el resultado sea reproducible.

```
def generar_jornada_aleatoria(seed=42):
    """
    Genera una jornada aleatoria (10 partidos) emparejando los 20 equipos sin repetición.
    Reproducible mediante seed.
    """
    equipos = [str(e) for e in dict_equipos.keys()]

    rng = np.random.default_rng(seed)
    perm = rng.permutation(equipos).tolist()  # para que salgan como str "normales"
    partidos = [(perm[2*i], perm[2*i+1]) for i in range(10)]
    return partidos

partidos_random = generar_jornada_aleatoria(seed=42)
partidos_random
```

```
[('Real Madrid CF', 'Granada CF'),
 ('Real Sociedad', 'RCD Espanyol'),
 ('RCD Mallorca', 'CD Leganés'),
 ('SD Eibar', 'Villarreal CF'),
 ('FC Barcelona', 'Deportivo Alavés'),
 ('Sevilla FC', 'RC Celta de Vigo'),
 ('Levante UD', 'Real Valladolid CF'),
 ('Atlético de Madrid', 'Real Betis Balompié'),
 ('Valencia CF', 'Athletic Club'),
 ('CA Osasuna', 'Getafe CF')]
```

2.11. Aplica el algoritmo al juego de datos generado.

Aplicamos el algoritmo exacto propuesto a la jornada aleatoria y mostramos:

- la asignación partido → franja,
- el desglose en tabla,
- la audiencia total obtenida.

```
sol_random, total_random = asignar_partidos_jornada(partidos_random)

mostrar_resultado_jornada(
    sol_random,
    algoritmo="Algoritmo exacto mejorado (backtracking sobre distribuciones + asignación óptima por ordenación)",
    nota="Jornada generada aleatoriamente para validar el método."
)
```

	Partido	Categorías	Horario	Base	CoefHorario	k_simult	CoefCoinc	Audiencia
7	Levante UD - Real Valladolid CF	B-C	D12	0.75	0.45	1	1.0	0.34
4	Atlético de Madrid - Real Betis Balompié	B-B	D16	0.90	0.75	1	1.0	0.68
2	Real Madrid CF - Granada CF	A-C	D18	1.00	0.85	1	1.0	0.85
1	FC Barcelona - Deportivo Alavés	A-B	D20	1.30	1.00	1	1.0	1.30
9	RCD Mallorca - CD Leganés	C-C	L20	0.47	0.40	1	1.0	0.19
6	SD Eibar - Villarreal CF	C-B	S12	0.75	0.55	1	1.0	0.41
5	Valencia CF - Athletic Club	B-B	S16	0.90	0.70	1	1.0	0.63
3	Sevilla FC - RC Celta de Vigo	B-B	S18	0.90	0.80	1	1.0	0.72
0	Real Sociedad - RCD Espanyol	A-B	S20	1.30	1.00	1	1.0	1.30
8	CA Osasuna - Getafe CF	C-B	V20	0.75	0.40	1	1.0	0.30

```
Algoritmo usado: Algoritmo exacto mejorado (backtracking sobre distribuciones + asignación óptima por ordenación)
Jornada generada aleatoriamente para validar el método.
Total: 6.72 (millones)
Conteo por franja: Counter({'S20': 1, 'D20': 1, 'D18': 1, 'S18': 1, 'D16': 1, 'S16': 1, 'S12': 1, 'D12': 1, 'V20': 1, 'L20': 1})
```

2.12. Enumera las referencias que has utilizado (si ha sido necesario) para llevar a cabo el trabajo.

Principales

- Manual de la asignatura *Algoritmos de optimización.*

- Ficheros CSV con los datos del problema (`equipos_categoria.csv`, `audiencia_categoria.csv`, `coef_horario.csv`, `coef_coincidencia.csv`).
- Documentación oficial de Python.
- Documentación de `pandas`.

Adicionales

- Kendall, G., Knust, S., Ribeiro, C. C., & Urrutia, S. (2010). *Scheduling in sports: An annotated bibliography. Computers & Operations Research, 37*(1), 1–19. Enlace
- Rasmussen, R. V., & Trick, M. A. (2008). *Round robin scheduling – a survey. European Journal of Operational Research, 188*(3), 617–636. Enlace

2.13. Describe brevemente las líneas de cómo crees que es posible avanzar en el estudio del problema. Ten en cuenta incluso posibles variaciones del problema y/o variaciones al alza del tamaño.

Si se quisiera llevar este trabajo más allá de una sola jornada, una ampliación natural sería pasar al calendario completo de la temporada. En ese caso ya no bastaría con decidir en qué franja va cada partido de una jornada concreta, sino que habría que tener en cuenta restricciones globales entre jornadas. Por ejemplo, repartir de forma equilibrada los horarios de máxima audiencia, evitar situaciones poco razonables de descanso —como jugar un lunes y volver a hacerlo el viernes siguiente— o controlar aspectos de equidad relacionados con partidos en casa y fuera, rachas o distribución de horarios.

En un escenario así, una forma razonable de mantener un enfoque exacto sería reformular el problema con técnicas de optimización más generales, como programación entera o programación con restricciones, y resolverlo con herramientas del tipo branch-and-bound o CP-SAT.

Sin salir del modelo actual, también habría margen para afinar la resolución exacta. Por ejemplo, se podría trabajar con alguna forma de descomposición por jornadas y hacer después un ajuste global, incorporar podas más fuertes en el backtracking mediante cotas superiores rápidas, o reutilizar cálculos mediante memoización cuando aparezcan distribuciones con estructuras parecidas.

Por último, el propio problema podría enriquecerse con variantes más realistas. Se podría hacer que la penalización por coincidencia dependiera también de la franja concreta, imponer reglas de equidad sobre cuántas veces aparece cada equipo o cada categoría en ciertos horarios destacados, o incluso añadir restricciones de descanso y desplazamientos si se dispusiera de información adicional.

3. Resolución del problema

3.1 Resolución exacta de una jornada

Aplicamos el **algoritmo exacto mejorado** a la jornada del enunciado.

Este algoritmo:

- reparte cuántos partidos van en cada franja (backtracking sobre distribuciones),
- y, para cada distribución, asigna los partidos de forma óptima emparejando audiencias base y multiplicadores ordenados.

```
# Jornada del enunciado (10 partidos)
partidos_enunciado = [
    ('RC Celta de Vigo', 'Real Madrid CF'),
    ('Valencia CF', 'Real Sociedad'),
    ('RCD Mallorca', 'SD Eibar'),
    ('Athletic Club', 'FC Barcelona'),
    ('CD Leganés', 'CA Osasuna'),
    ('Villarreal CF', 'Granada CF'),
    ('Deportivo Alavés', 'Levante UD'),
    ('RCD Espanyol', 'Sevilla FC'),
    ('Real Betis Balompié', 'Real Valladolid CF'),
    ('Atlético de Madrid', 'Getafe CF'),
]

sol_enunciado, _ = asignar_partidos_jornada(partidos_enunciado)

mostrar_resultado_jornada(
    sol_enunciado,
    algoritmo="Algoritmo exacto mejorado (backtracking sobre distribuciones + asignación óptima por ordenación)"
)
```

	Partido	Categorías	Horario	Base	CoefHorario	k_simult	CoefCoinc	Audiencia
7	Real Betis Balompié - Real Valladolid CF	B-C	D12	0.75	0.45	1	1.0	0.34
4	RCD Espanyol - Sevilla FC	B-B	D16	0.90	0.75	1	1.0	0.68
2	Athletic Club - FC Barcelona	B-A	D18	1.30	0.85	1	1.0	1.10
1	Valencia CF - Real Sociedad	B-A	D20	1.30	1.00	1	1.0	1.30
9	CD Leganés - CA Osasuna	C-C	L20	0.47	0.40	1	1.0	0.19
6	Villarreal CF - Granada CF	B-C	S12	0.75	0.55	1	1.0	0.41
5	Atlético de Madrid - Getafe CF	B-B	S16	0.90	0.70	1	1.0	0.63
3	Deportivo Alavés - Levante UD	B-B	S18	0.90	0.80	1	1.0	0.72
0	RC Celta de Vigo - Real Madrid CF	B-A	S20	1.30	1.00	1	1.0	1.30
8	RCD Mallorca - SD Eibar	C-C	V20	0.47	0.40	1	1.0	0.19

```
Algoritmo usado: Algoritmo exacto mejorado (backtracking sobre distribuciones + asignación óptima por ordenación)
Total: 6.86 (millones)
Conteo por franja: Counter({'S20': 1, 'D20': 1, 'D18': 1, 'S18': 1, 'D16': 1, 'S16': 1, 'S12': 1, 'D12': 1, 'V20': 1, 'L20': 1})
```

3.2. Resolución por fuerza bruta para casos pequeños

Aplicamos ahora una **demo de fuerza bruta exacta** sobre un subconjunto reducido de partidos.

Se usa un caso pequeño porque la fuerza bruta crece exponencialmente con el número de partidos:

- con 7 partidos sigue siendo asumible,
- con 8 partidos el tiempo ya aumenta de forma notable.

Esta demo se incluye solo como comprobación del método exhaustivo.

```
n_partidos = 8
partidos_demo = partidos_enunciado[:n_partidos]

sol_demo, _ = asignar_partidos_jornada_fuerza_bruta(partidos_demo)

mostrar_resultado_jornada(
    sol_demo,
    algoritmo="Fuerza bruta exacta (backtracking completo sobre todas las asignaciones)",
    nota=f"NOTA: Demo reducida con {n_partidos} partidos para mantener un tiempo de ejecución razonable."
)
```

	Partido	Categorías	Horario	Base	CoefHorario	k_simult	CoefCoinc	Audiencia
7	RCD Espanyol - Sevilla FC	B-B	D16	0.90	0.75	1	1.0	0.68
1	Valencia CF - Real Sociedad	B-A	D18	1.30	0.85	1	1.0	1.10
3	Athletic Club - FC Barcelona	B-A	D20	1.30	1.00	1	1.0	1.30
4	CD Leganés - CA Osasuna	C-C	L20	0.47	0.40	1	1.0	0.19
5	Villarreal CF - Granada CF	B-C	S16	0.75	0.70	1	1.0	0.52
6	Deportivo Alavés - Levante UD	B-B	S18	0.90	0.80	1	1.0	0.72
0	RC Celta de Vigo - Real Madrid CF	B-A	S20	1.30	1.00	1	1.0	1.30
2	RCD Mallorca - SD Eibar	C-C	V20	0.47	0.40	1	1.0	0.19

```
Algoritmo usado: Fuerza bruta exacta (backtracking completo sobre todas las asignaciones)
NOTA: Demo reducida con 8 partidos para mantener un tiempo de ejecución razonable.
Total: 6.0 (millones)
Conteo por franja: Counter({'S20': 1, 'D18': 1, 'V20': 1, 'D20': 1, 'L20': 1, 'S16': 1, 'S18': 1, 'D16': 1})
```